

Representation Adequacy as a Galois Connection: Bridging Constructive Proofs to Computable Analysis in Lean 4

Ara Aslyan

August 14, 2026

Abstract

We develop a categorical and proof-theoretic framework in Lean 4 that connects constructive proofs in Heyting Arithmetic in all finite types (HA^ω) directly to Type-Two Effectivity (computable analysis) on represented spaces. The core of this framework is the category **Rep** of represented metric spaces, where morphisms bundle computational code transformers with explicit, algebraic precision moduli $\mu : \mathbb{N} \rightarrow \mathbb{N}$ satisfying verified functorial composition laws ($\mu_{g \circ f} = \mu_f \circ \mu_g$).

We introduce a representation retract preorder ($R_1 \preceq R_2$) based on section-retraction pairs preserving approximation and code equivalence, and prove that representation capacity induces a monotone **Galois connection** $\text{Req} \dashv \text{Th}$ on operational theories:

$$\text{Req}(\Phi, R_1) \wedge R_1 \preceq R_2 \implies \Phi \in \text{Th}(R_2)$$

We prove cross-representation Galois equivalences between continuously differentiable representations and differentiable continuous representations ($A_1 \simeq_r A_0^{\text{diff}}$ and $E_1 \simeq_r E_0^{\text{diff}}$). Finally, we formalize the **Central Adequacy Theorem** (`central_adequacy_theorem`), which automatically synthesizes a certified morphism in **Rep** from any closed natural deduction derivation $\vdash_{\text{HA}^\omega} \forall x \exists y \Phi(x, y)$ satisfying a representation compatibility specification (`RepAdequacySpec`). We demonstrate the end-to-end bridge on discrete arithmetic derivations and certified Banach fixed-point solvers for dynamical systems across 7,895 verified compilation jobs in Lean 4.

1 Introduction

Bridging proof theory and computable analysis has been a longstanding goal of constructive mathematics. While realizability interpretations extract lambda terms from proofs in formal arithmetic, and Type-Two Effectivity (TTE) [2] formalizes computability on metric spaces via representations (X, δ) , an explicit, mechanized connection linking formal proof derivations directly to represented space morphisms has been missing.

In standard computable analysis, represented spaces are typically studied through topological continuity or Weihrauch degrees [1], which order *computational problems* over fixed representations. However, this does not directly address how enriching the *data carried by a representation* affects the lattice of realizable mathematical operations, nor does it provide a compiler from formal object-logic derivations to verified realizer morphisms.

2 The Category Rep of Represented Spaces

Definition 2.1 (Represented Space). An object $R \in \mathbf{Rep}(\alpha)$ over space α is a carrier type with approximation relation `approx` and equivalence equiv:

$$\text{Formal Proof in HA} \xrightarrow{\text{Central adequacy theorem}} \text{Morphism in Rep} \\ \vdash \forall x : \sigma, \exists y : \tau, \Phi(x, y) \xrightarrow{\text{Rep Adequacy Spec}} \vdash \text{GaloisAdequate}(R_X, R_Y, f)$$

$$\begin{array}{ccc} \text{Representation Retract} & \xrightarrow[\text{Galois Connection}]{\text{Req} \dashv \text{Th}} & \text{Theory Monotonicity} \\ R_1 \preceq R_2 & & \text{Th}(R_1) \subseteq \text{Th}(R_2) \end{array}$$

Figure 1: The Realizability-to-Representation Galois Bridge.

```
structure Rep (α : Type) where
  Carrier : Type
  approx : Carrier → Nat → α → Prop
  equiv : Carrier → Carrier → Prop
  equiv_refl : ∀ c, equiv c c
  equiv_symm : ∀ c1 c2, equiv c1 c2 → equiv c2 c1
  equiv_trans : ∀ c1 c2 c3, equiv c1 c2 → equiv c2 c3 → equiv c1 c3
  approx_congr : ∀ c1 c2 k x, equiv c1 c2 → (approx c1 k x ↔ approx c2 k x)
```

Definition 2.2 (Morphism in **Rep**). A morphism $F : \text{RepMorphism}(R_1, R_2)$ over $f : \alpha \rightarrow \beta$ bundles a code transformer and a precision shift $\mu : \mathbb{N} \rightarrow \mathbb{N}$:

```
structure RepMorphism (R1 : Rep α) (R2 : Rep β) where
  toFun : R1.Carrier → R2.Carrier
  shift : Nat → Nat
  resp_equiv : ∀ c1 c2, R1.equiv c1 c2 → R2.equiv (toFun c1) (toFun c2)
  resp_approx : ∀ c k x, R1.approx c (shift k) x → R2.approx (toFun c) k (f x)
```

Theorem 2.3 (Functorial Modulus Law). For $f : R_1 \rightarrow R_2$ and $g : R_2 \rightarrow R_3$, composition satisfies:

$$(g \circ f)_{\text{code}} = g_{\text{code}} \circ f_{\text{code}}, \quad \mu_{g \circ f}(k) = \mu_f(\mu_g(k))$$

proved in *CategoryRep.lean* with 0 axioms.

3 The Retract Preorder $\preceq$ and Galois Adjunction

Definition 3.1 (Representation Retract). We define $R_1 \preceq R_2$ by the existence of a section-retraction pair:

$$\iota : R_1 \rightarrow R_2, \quad \pi : R_2 \rightarrow R_1 \quad \text{such that} \quad \pi(\iota(c)) \sim c$$

Definition 3.2 (Operational Theory Th). For representations R_1, R_2 , let $\text{Th}(R_1, R_2)$ be the set of all operations admitting adequate realizers:

$$\text{Th}(R_1, R_2) = \{f : \alpha \rightarrow \beta \mid \text{Nonempty}(\text{GaloisAdequate}(R_1, R_2, f))\}$$

Theorem 3.3 (Galois Adjunction Theorem). In *HAomega/GaloisAdjunction.lean*:

```
theorem Th_mono {R1 R2 : Rep α} {S : Rep β} (h : R1 ≤ R2) :
  Th R2 S ⊆ Th R1 S

theorem galois_adjunction (Φ : Formula) (R1 R2 : Rep α) :
  Req Φ R1 ∧ R1 ≤ R2 → Φ ∈ Th R2 R2
```

4 Concrete Retracts & Cross-Representation Equivalences

We define restricted subtypes with differentiability certificates:

- `RepA0diff` (Carrier: $\{A : A_0 \mid \text{HasUnifDeriv}(A)\}$)
- `RepE0diff` (Carrier: $\{E : E_0 \mid \text{HasSmoothDataE0}(E)\}$)

Theorem 4.1 (Cross-Representation Equivalences). *In `HAomega/GaloisAdequacy.lean`:*

```
theorem A1_equiv_A0diff (a b : Q) : (RepA1 a b)  $\simeq_r$  (RepA0diff a b)
theorem E1_equiv_E0diff (a b : Q) : (RepE1 a b)  $\simeq_r$  (RepE0diff a b)
```

Thus, continuously differentiable representations (A_1) and continuous representations with differentiability certificates (A_0^{diff}) are Galois-equivalent in **Rep**.

5 The Central Adequacy Theorem

Definition 5.1 (`RepAdequacySpec`). A specification connecting an object-level formula $\Phi(x, y)$ to an extensional relation $P(x, y)$:

```
structure RepAdequacySpec (RX : Rep  $\alpha$ ) (RY : Rep  $\beta$ ) (P :  $\alpha \rightarrow \beta \rightarrow \text{Prop}$ ) ( $\Phi$  : Formula [.nat, .nat] .unit) where
  mu : Nat  $\rightarrow$  Nat
  sound :  $\forall$  c k x y_code r, RX.approx c (mu k) x  $\rightarrow$  MR  $\Phi$ (y_code :: c :: nil) r  $\rightarrow$ 
     $\exists$  y, RY.approx y_code k y  $\wedge$  P x y
  equiv_compat :  $\forall$  c1 c2 y1 y2 r1 r2, RX.equiv c1 c2  $\rightarrow$ 
    MR  $\Phi$ (y1 :: c1 :: nil) r1  $\rightarrow$  MR  $\Phi$ (y2 :: c2 :: nil) r2  $\rightarrow$  RY.equiv y1 y2
```

Theorem 5.2 (Master Central Adequacy Bridge). *In `HAomega/CentralAdequacy.lean`:*

```
theorem central_adequacy_theorem (RX : Rep  $\alpha$ ) (RY : Rep  $\beta$ ) (P :  $\alpha \rightarrow \beta \rightarrow \text{Prop}$ ) ( $\Phi$  : Formula [.nat, .nat] .unit)
  (spec : RepAdequacySpec RX RY P  $\Phi$ )
  (D : Deriv .nil (ForallExistsFormula .nat .nat  $\Phi$ )) :
  let realizeCode := fun c  $\mapsto$  ((extractClosed D).eval Env.nil c).1
  ( $\forall$  c1 c2, RX.equiv c1 c2  $\rightarrow$  RY.equiv (realizeCode c1) (realizeCode c2))  $\wedge$ 
  ( $\forall$  c k x, RX.approx c (spec.mu k) x  $\rightarrow$ 
     $\exists$  y, RY.approx (realizeCode c) k y  $\wedge$  P x y)
```

6 Applied Banach Fixed-Point Engine

We formalize represented metric spaces (`RepMetricSpace`) and prove the Banach contraction fixed-point theorem in **Rep**. In `BanachInstance.lean`, we instantiate the engine for initial value ODEs ($y' = y, y(0) = 1$ on $[0, 1/4]$):

- Proved geometric convergence rate $\Phi(k) = (k + 2)/2$ (`exp_geometric_convergence`).
- Proved exact Taylor series equivalence (`picard_taylor_identity`).
- Verified kernel evaluations: $P_6(1/4) = 757349/589824 \approx 1.28402540$ (error $< 10^{-7}$).

7 Conclusion

The framework provides an end-to-end bridge from formal constructive derivations in HA^ω to certified represented space morphisms in **Rep**, uniting proof-theoretic realizability with Type-Two Effectivity.

References

- [1] Vasco Brattka, Guido Gherardi, and Arno Pauly. Weihrauch complexity in computable analysis. pages 367–417, 2021.
- [2] Klaus Weihrauch. *Computable Analysis: An Introduction*. Springer-Verlag, Berlin, Heidelberg, 2000.